

# Trading Agentic

Cómo funciona tu sistema, de punta a punta — sin tantos tecnicismos.

## Frontend

Next.js 16  
Vercel  
Dashboard + APIs

## Backend

Python FastAPI  
Dokploy VPS  
Motor cuantitativo

## Datos & IA

Supabase (DB)  
Gemini + Claude  
Binance Testnet

## ¿Qué hace este sistema en una frase?

Investiga el mercado con IA, decide cuándo abrir/cerrar operaciones siguiendo una estrategia cuantitativa, ejecuta en Binance Testnet con un kill-switch, y te avisa por Telegram con botones de aprobación.

## Las 3 piezas que tenés que recordar

### ■ 1. Cerebro de IA (research + estrategia)

- Lee fuentes, papers y noticias
- Genera 'guías de trading' y propone ajustes de parámetros
- Modelos: Gemini (rápido) + Claude (Strategist Agent — Fase 1)

### ■ 2. Motor cuantitativo (Python / FastAPI)

- Loop de 60s: indicadores técnicos, regímenes, señales
- Aplica reglas: cooldown, ATR, riesgo, Chandelier Exit
- Habla con Binance Testnet para abrir/cerrar posiciones

### ■ 3. Operador humano (vos)

- Mirás el dashboard en Vercel
- Aprobás/rechazás cambios de configuración por Telegram
- Activás el kill-switch (TRADING\_ENABLED) cuando hace falta

## Workflow completo — el viaje de una operación

Así se mueve la información desde que el sistema mira el mercado hasta que una orden queda ejecutada y vos recibís el aviso.

### ■ 1. Captura de datos de mercado

- Binance Testnet → velas (OHLCV), order book
- Binance Futures (read-only) → funding rate, open interest

### ■ 2. Análisis cuantitativo (cada 60s)

- Indicadores: RSI, ADX, ATR, EMAs, entropía
- Detecta régimen: trend / range / chop

### ■ 3. Cerebro IA decide ajustes (1x/día)

- Strategist Agent (Claude) propone cambios de parámetros
- Bounds SSOT — nunca puede pedir valores fuera de rango seguro

### ■ 4. Aprobación por Telegram

- Te llega un mensaje con botones [Aprobar] [Rechazar]
- Token timing-safe — solo vos podés aprobar

### ■ 5. Generación de señal

- Si hay setup válido y no estás en cooldown → señal BUY/SELL
- Calcula tamaño con ATR + riesgo configurado

### ■ 6. Ejecución en Binance Testnet

- Orden con SL/TP basados en ATR
- Partial exit 50% al 1R + Chandelier trailing

### ■ 7. Registro y aviso

- Posición guardada en Supabase
- Telegram te avisa: entrada, salida parcial, cierre

Si en cualquier paso `TRADING_ENABLED=false`, el sistema sigue analizando pero **no ejecuta nada**. Es tu freno de mano.

## Herramientas que usa el sistema

Cada herramienta hace una cosa concreta. Si algo falla, ya sabés a quién mirar.

| Categoría | Herramienta                   | Para qué sirve                           |
|-----------|-------------------------------|------------------------------------------|
| Frontend  | Next.js 16 (App Router)       | Dashboard, formularios, API routes       |
|           | React 19 + Tailwind 4         | UI del operador                          |
|           | shadcn/ui + Lucide            | Componentes y íconos                     |
|           | Vercel                        | Hosting + Cron jobs (cleanup, watchdog)  |
| Backend   | Python 3.12 + FastAPI         | Motor cuantitativo y API trading         |
|           | Pandas / NumPy / SciPy        | Procesamiento numérico de velas          |
|           | Pandas-TA                     | Indicadores técnicos (RSI, ADX, ATR...)  |
|           | httpx (async)                 | Cliente HTTP para Binance Futures        |
|           | Dokploy (VPS)                 | Despliegue del backend en Docker         |
| Datos     | Supabase Postgres             | DB: posiciones, configs, logs            |
|           | Supabase Auth                 | Login del dashboard                      |
| IA        | Google Gemini (Vercel AI SDK) | Source / Reader / Synthesis / Chat       |
|           | Claude Agent SDK (Python)     | Strategist Agent — Fase 1                |
|           | LangGraph (en retiro)         | Pipeline diario antiguo, en decommission |
| Trading   | Binance Spot Testnet          | Ejecución real de órdenes                |
|           | Binance Futures (read-only)   | Funding rate + open interest             |
| Avisos    | Telegram Bot API              | Notifs + aprobaciones inline             |
| Calidad   | pytest + pytest-asyncio       | Tests del backend (56+)                  |
|           | ESLint + TypeScript strict    | Linter y tipos del frontend              |
|           | Playwright                    | Tests integrales                         |
| CI/CD     | GitHub Actions                | Lint + typecheck + tests + build         |

## Los 3 ciclos del sistema

El sistema corre en paralelo con tres relojes distintos. Cada uno tiene su responsabilidad y nunca se pisan.

### ■ Ciclo rápido — cada 60 segundos

- Lee precios y recalcula indicadores
- Evalúa entradas/salidas, partial exit, trailing
- Ejecuta órdenes en Binance si hay señal

### ■ Ciclo medio — cada 5 minutos (Vercel Cron)

- Limpia configs pendientes vencidas
- Watchdog: chequea que el backend Python esté vivo
- Fallback: si el backend cae, ejecuta análisis básico

### ■ Ciclo lento — 1 vez al día (Strategist Agent)

- Investiga mercado: regímenes, papers, noticias
- Propone nuevos parámetros (RSI, ADX, riesgo, etc.)
- Te pide aprobación por Telegram → vos decidís

## Las barreras de seguridad

### ■ Kill switch

- `TRADING_ENABLED=false` detiene toda ejecución sin tocar código.

### ■ Bounds SSOT

- La IA nunca puede salirse del rango seguro de parámetros.

### ■ Aprobación humana

- Cambios de config requieren tu OK por Telegram.

### ■ Solo Testnet

- Binance Spot Testnet — sin plata real hasta que lo decidas.

### ■ SSRF + Auth

- API routes protegidas, middleware Supabase en el dashboard.

## Cheat sheet — comandos del día a día

| Acción                  | Comando                                                  |
|-------------------------|----------------------------------------------------------|
| Levantar frontend       | <code>pnpm dev</code>                                    |
| Build frontend          | <code>pnpm build</code>                                  |
| Lint + tipos            | <code>pnpm lint &amp;&amp; pnpm typecheck</code>         |
| Levantar backend Python | <code>uvicorn app.main:app --reload --port 8000</code>   |
| Tests backend           | <code>pytest tests/ -q --tb=short</code>                 |
| Refrescar contexto KB   | <code>python scripts/refresh-market-context.py</code>    |
| Smoke Binance Futures   | <code>python backend/scripts/smoke_derivatives.py</code> |
| Chequear proxy drift    | <code>python backend/scripts/check_proxy_drift.py</code> |

### Variables de entorno que importan

| Variable                         | Para qué                                  |
|----------------------------------|-------------------------------------------|
| <b>TRADING_ENABLED</b>           | Kill switch. Default: false               |
| <b>BINANCE_TESTNET_API_KEY</b>   | Llave del exchange (testnet)              |
| <b>GOOGLE_AI_API_KEY</b>         | Gemini (research + chat)                  |
| <b>TELEGRAM_BOT_TOKEN</b>        | Notifs y botones de aprobación            |
| <b>STRATEGIST_APPROVAL_TOKEN</b> | Token para aprobar configs                |
| <b>CRON_SECRET</b>               | Auth de los cron jobs Vercel              |
| <b>LANGGRAPH_DAILY_ENABLED</b>   | Apaga el pipeline viejo cuando esté listo |
| <b>PARTIAL_EXIT_ENABLED</b>      | Activa toma parcial al 1R                 |

Este documento es un mapa, no un manual exhaustivo. Para detalles técnicos: [docs/superpowers/specs/](#) y [docs/knowledge-base/](#).